

Mejoras en las heurísticas de eliminación de variables de OpenMarkov

CISIAD — UNED

Marzo 2026

Índice

1. Introducción	1
2. Corrección de CanoMoralElimination	2
2.1. Error 1: la lista de cliques maximales siempre estaba vacía	2
2.2. Error 2: getMaxClique podía devolver null	2
3. Nueva heurística: WeightedMinFill	3
3.1. Motivación	3
3.2. Función de coste	3
3.3. Criterio de desempate	3
3.4. Implementación	4
4. Heurísticas con búsqueda anticipada (<i>lookahead</i>)	4
4.1. Motivación	4
4.2. Algoritmo	4
4.3. Clase base: LookaheadHeuristic	4
4.4. LookaheadMinFill	5
4.5. LookaheadMinCliqueSize	5
4.6. LookaheadCanoMoral	5
5. Evaluación experimental	5
5.1. Alcance del experimento	5
5.2. Metodología	5
5.3. Análisis de resultados	6
6. Diagrama de clases	7
7. Resumen de ficheros modificados	7

1. Introducción

Las heurísticas de eliminación de variables determinan el orden en que se eliminan las variables al triangular el grafo moral de una red bayesiana. Un buen orden de eliminación minimiza el *treewidth*, es decir, el tamaño del clique más grande generado durante la triangulación menos uno. Cuanto menor es el *treewidth*, menor es el coste de la inferencia exacta.

OpenMarkov proporciona varias heurísticas de eliminación. En este documento se describen tres grupos de cambios realizados:

1. Corrección de dos errores en **CanoMoralElimination**.
2. Implementación de una nueva heurística greedy: **WeightedMinFill**.
3. Implementación de un marco genérico de búsqueda anticipada (*lookahead*) y tres instancias concretas: **LookaheadMinFill**, **LookaheadMinCliqueSize** y **LookaheadCanoMoral**.

2. Corrección de CanoMoralElimination

La clase **CanoMoralElimination** implementa el algoritmo de triangulación heurística de Cano y Moral [1]. La métrica que utiliza es:

$$H_6(X_i) = \frac{S(X_i)}{C(X_i)} \quad (1)$$

donde $S(X_i)$ es el tamaño del clique creado al eliminar X_i (producto de las cardinalidades de X_i y sus vecinos), y $C(X_i)$ es la suma de los tamaños de todos los cliques maximales del subgrafo inducido por X_i y sus vecinos. Se elimina la variable con menor H_6 .

Se detectaron dos errores independientes:

2.1. Error 1: la lista de cliques maximales siempre estaba vacía

En el método **sumCliqueSizes**, el bucle que evita añadir cliques ya subsumidos usaba la condición incorrecta:

Listing 1: Condición original (incorrecta)

```

1 for (i = 0; i < numMaximalCliques && !maximalCliques.get(i).containsAll(
    maximalClique); i++) { }
2 if (i < numMaximalCliques) { // BUG: debería ser i == numMaximalCliques
3     maximalCliques.add(maximalClique);
4 }

```

La condición `i < numMaximalCliques` es cierta cuando el bucle terminó *antes de recorrer todos los cliques*, es decir, cuando encontró uno que sí subsume al nuevo. La condición *correcta* para añadir el clique es que el bucle haya llegado al final sin encontrar ningún subsumisor:

Listing 2: Condición corregida

```

1 if (i == numMaximalCliques) {
2     maximalCliques.add(maximalClique);
3 }

```

Como consecuencia del bug original, **sumCliqueSizes** siempre devolvía 0, lo que hacía que H_6 fuera siempre **MAX_VALUE** para todos los nodos, y la heurística siempre elegía el último candidato de la lista (comportamiento arbitrario).

2.2. Error 2: getMaxClique podía devolver null

El método **getMaxClique** calcula el clique maximal que contiene una semilla dada. Su variable local **maxClique** se inicializaba a **null**:

Listing 3: Inicialización original (incorrecta)

```

1 Set<Node> maxClique = null;
2 long maxCliqueSize = 0;
3 for (Node node : candidates) {
4     ...
5     if (maxCliqueCandidateSize > maxCliqueSize) {
6         maxClique = maxCliqueCandidate;
7     }
8 }
9 return maxClique; // devuelve null si candidates estaba vacío

```

Cuando no había ningún candidato para expandir la semilla (la semilla ya es un clique maximal), el método devolvía `null`. Esto provocaba una `NullPointerException` en `sumCliqueSizes` al comparar con `containsAll`. La corrección es inicializar `maxClique` con la semilla misma, que es el clique maximal cuando no existe ninguna expansión posible:

Listing 4: Inicialización corregida

```

1 Set<Node> maxClique = seed;
2 long maxCliqueSize = getCliqueSize(seed);

```

Los dos bugs eran independientes pero se enmascaraban mutuamente: mientras existió el primer bug, `getMaxClique` nunca era llamado desde `sumCliqueSizes` con una semilla que requiriera regresar `null`, por lo que el segundo bug no se manifestaba. Al corregir el primero, el segundo quedó expuesto.

3. Nueva heurística: WeightedMinFill

3.1. Motivación

La heurística `MinimalFillIn` cuenta el número de arcos de relleno (*fill-in*) que generaría la eliminación de una variable, sin tener en cuenta el tamaño de las tablas de probabilidad asociadas a esas variables. `WeightedMinFill` corrige este sesgo ponderando cada arco de relleno por las cardinalidades de los dos extremos del arco.

3.2. Función de coste

Para una variable X con vecinos $\mathcal{N}(X)$:

$$\text{cost}(X) = \sum_{\substack{Y, Z \in \mathcal{N}(X) \\ Y \neq Z \\ (Y, Z) \notin E}} \text{card}(Y) \cdot \text{card}(Z) \quad (2)$$

donde la suma recorre todos los pares de vecinos de X que *no* están ya conectados por un arco (los que formarían arcos de relleno al eliminar X), y $\text{card}(V)$ es el número de estados de la variable V .

3.3. Criterio de desempate

En caso de empate en el coste, se da prioridad a la variable con *menos vecinos actuales*; si persiste el empate, se elige la variable de menor nombre (orden lexicográfico, tolerante a `null`).

3.4. Implementación

La clase extiende `EliminationHeuristic` y mantiene una copia del grafo (`graphCopy`) actualizada tras cada eliminación:

- `getVariableToDelete()`: recorre los candidatos, calcula la ecuación (2) para cada uno mediante `weightedFillInCost(Node)` y devuelve el de menor coste.
- `afterEditExecutes(PNEdit)`: añade los arcos de relleno en `graphCopy` y elimina el nodo correspondiente.

4. Heurísticas con búsqueda anticipada (*lookahead*)

4.1. Motivación

Las heurísticas greedy toman decisiones de eliminación mirando únicamente el coste inmediato. Una estrategia de búsqueda anticipada evalúa también las consecuencias de esa elección en pasos futuros, con la esperanza de evitar decisiones localmente buenas pero globalmente costosas.

4.2. Algoritmo

Sea G el grafo moral actual, X un candidato a eliminar y d la profundidad de búsqueda. La función de puntuación es:

$$\text{score}(G, X, d) = \begin{cases} c_0(G, X) & \text{si } d = 0 \\ c_0(G, X) + \min_{Y \in \text{beam}(G \setminus X)} \text{score}(G \setminus X, Y, d - 1) & \text{si } d > 0 \end{cases} \quad (3)$$

donde:

- $c_0(G, X)$ es el *coste inmediato* de eliminar X (definido por la subclase concreta),
- $G \setminus X$ denota el grafo resultante de eliminar X y añadir los arcos de relleno necesarios,
- $\text{beam}(G')$ es el subconjunto de hasta `beamWidth` candidatos de menor coste inmediato en G' (búsqueda en haz).

Los valores por defecto son `depth = 2` y `beamWidth = 3`.

4.3. Clase base: `LookaheadHeuristic`

Para evitar la duplicación de código, se diseñó una clase abstracta `LookaheadHeuristic` que implementa toda la infraestructura del algoritmo (3). Las subclases solo tienen que implementar:

Listing 5: Método plantilla de `LookaheadHeuristic`

```
1 protected abstract double immediateCost(boolean[][] adj, boolean[] active,  
2 int x, int[] cardinalities);
```

donde:

- `adj`: la matriz de adyacencia actual (no se muta),
- `active`: máscara booleana de los nodos aún presentes,
- `x`: índice del nodo a evaluar,
- `cardinalities`: número de estados de cada nodo.

La búsqueda anticipada opera exclusivamente sobre una matriz `boolean[][]` ligera, sin construir copias de `ProbNet`. Esto hace que el coste adicional sea manejable. La copia real del grafo (`graphCopy`) solo se actualiza en `afterEditExecutes`, es decir, una vez por eliminación real, no durante la búsqueda.

4.4. LookaheadMinFill

Coste inmediato: fill-in ponderado (ecuación (2)), calculado directamente sobre la matriz booleana.

$$c_0^{\text{LMF}}(G, X) = \sum_{\substack{Y, Z \in \mathcal{N}(X) \\ (Y, Z) \notin E}} \text{card}(Y) \cdot \text{card}(Z) \quad (4)$$

4.5. LookaheadMinCliqueSize

Coste inmediato: tamaño del clique que formaría X con sus vecinos.

$$c_0^{\text{LCS}}(G, X) = |\mathcal{N}(X)| + 1 \quad (5)$$

4.6. LookaheadCanoMoral

Coste inmediato: la misma métrica H_6 de la ecuación (1), reimplementada sobre la representación `boolean[][]`. El método `getMaxClique` trabaja sobre máscaras booleanas en lugar de conjuntos de `Node`, con la misma lógica recursiva que en `CanoMoralElimination` pero sin dependencias de objetos del dominio.

$$c_0^{\text{LCM}}(G, X) = \frac{S(X)}{C(X)} \quad (\text{cf. ecuación (1)}) \quad (6)$$

5. Evaluación experimental

5.1. Alcance del experimento

La heurística `LookaheadCanoMoral` no se incluye en la comparación experimental. Su función de coste inmediato requiere encontrar cliques maximales en el subgrafo de vecinos mediante un algoritmo recursivo (`getMaxClique`), cuyo coste crece exponencialmente con el número de vecinos. En redes grandes, el tiempo de evaluación hace que la heurística no termine en un plazo razonable. Las demás siete heurísticas sí se comparan.

5.2. Metodología

Se comparan las siete heurísticas sobre las redes bayesianas disponibles en el repositorio de OpenMarkov, filtradas para incluir solo redes con potenciales de tipo tabla y excluyendo las versiones con sufijo -1-0 (que son duplicadas). El repositorio contiene actualmente 7 redes válidas tras aplicar estos filtros. Para cada par (red, heurística):

- Se construye el árbol de unión mediante `HuginForest`.
- Se registran el *treewidth* ($= \max_c |c| - 1$, donde $|c|$ es el número de variables del clique c) y la *suma del árbol de unión* ($= \sum_c \prod_{V \in c} \text{card}(V)$), que es el coste real de la inferencia.
- El tiempo máximo por par es 5 minutos. Si se supera, se registra TIMEOUT.

La ejecución se paraleliza usando un pool de `Runtime.getRuntime().availableProcessors()` hilos. La estrategia es fila a fila: las H heurísticas de una misma red se ejecutan en paralelo; los resultados se recogen antes de cargar la siguiente red. Así el pico de memoria se limita a H copias de una red en lugar de $N \times H$ simultáneas.

Red	Vars	SimpleElim	MinFillIn	MinClique	CanoMoral	WgtMinFill	LAMinFill	LAMinClique
BN-nasonet.pgm	111	12	7	5	4	7	7	5
BN-hepar.pgm	71	17	5	5	6	5	6	6
BN-prostanet.pgm	47	12	4	4	5	5	5	4
BN-alarm.pgm	37	3	2	2	3	2	2	2
BN-asia.pgm	8	2	2	2	2	2	2	2
BN-two-diseases-naive.pgm	8	1	1	1	4	1	1	1
BN-one-disease.pgm	2	1	1	1	1	1	1	1

Tabla 1: Treewidth obtenido por cada heurística sobre las redes bayesianas del repositorio (versiones canónicas; valor más bajo es mejor). En negrita el mejor resultado de cada fila.

Red	Vars	SimpleElim	MinFillIn	MinClique	CanoMoral	WgtMinFill	LAMinFill	LAMinClique
BN-nasonet.pgm	111	78.53M	109.6K	33.3K	18.6K	163.9K	98.5K	34.3K
BN-hepar.pgm	71	28.47M	713	680	1500	692	628	640
BN-prostanet.pgm	47	40.6K	296	310	620	436	436	352
BN-alarm.pgm	37	508	404	343	669	232	358	377
BN-asia.pgm	8	32	32	32	32	32	32	32
BN-two-diseases-naive.pgm	8	42	42	42	66	42	42	42
BN-one-disease.pgm	2	4	4	4	4	4	4	4

Tabla 2: Suma del árbol de unión (producto de cardinalidades de cada clique sumado sobre todos los cliques). Mide el coste real de la inferencia. Valores en miles (K) o millones (M) cuando corresponde. En negrita el mejor resultado de cada fila.

5.3. Análisis de resultados

Las redes del repositorio son relativamente pequeñas (8–111 variables); los resultados deben interpretarse con esa cautela.

Treewidth. `CanoMoral` obtiene el menor treewidth en la red más grande (nasonet, 111 variables: treewidth 4 frente a 5 del siguiente). `LookaheadMinCliqueSize` es la única heurística lookahead que iguala ese resultado, y también es la mejor en prostanet. Sin embargo, `CanoMoral` se comporta de forma patológica en `BN-two-diseases-naive`: obtiene treewidth 4 mientras todas las demás obtienen 1. Esto sugiere que la métrica H6 tiene un caso desfavorable en redes con una topología determinada.

Suma del árbol de unión. Este indicador es más sensible a las diferencias. `CanoMoral` es la mejor en nasonet (18.6K frente a 33.3K de MinClique), pero la peor en alarm (669 frente a 232 de WeightedMinFill) y en two-diseases-naive (66 frente a 42 del resto). `WeightedMinFill` destaca como la mejor en alarm. `LookaheadMinFill` mejora a su versión greedy en hepar (628 frente a 713) pero queda por detrás en prostanet y alarm.

La búsqueda anticipada puede ser peor que su versión greedy. Esto se observa en hepar ($\text{LAMinClique} > \text{MinClique}$ para la suma) y alarm (ambas versiones lookahead superadas por WeightedMinFill). Las razones son:

1. **Truncamiento del haz:** el haz de anchura 3 puede descartar buenos caminos.
2. **Horizonte corto:** con profundidad 2, el algoritmo no ve el impacto más allá de dos pasos.

3. **Objetivo inadecuado:** la función score minimiza la *suma* de costes futuros, mientras que el treewidth depende del *máximo* de los tamaños de clique.
4. **Balance implícito:** las heurísticas greedy clásicas mantienen un reparto equilibrado de vecinos que la búsqueda anticipada puede romper.

6. Diagrama de clases

La figura 1 muestra la jerarquía de heurísticas de eliminación resultante. Las clases en naranja han sido corregidas; las clases en verde son nuevas heurísticas greedy; las clases en azul pertenecen al nuevo marco de búsqueda anticipada. El diagrama fue generado con PlantUML a partir de `diagrama_clases.puml`.

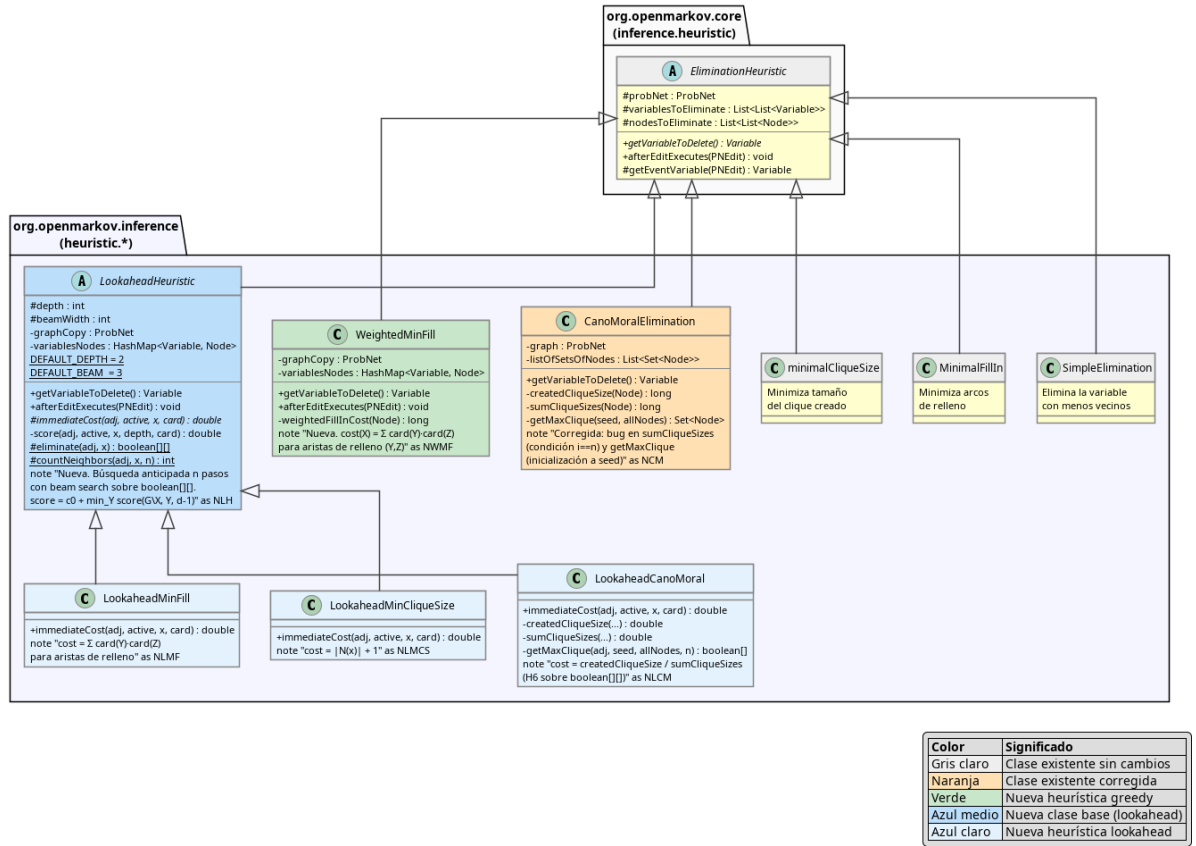


Figura 1: Diagrama de clases de las heurísticas de eliminación de OpenMarkov tras las modificaciones descritas en este documento.

7. Resumen de ficheros modificados

Referencias

- [1] A. Cano y S. Moral. Heuristic algorithms for the triangulation of graphs. En *Proceedings of IPMU'94*, págs. 298–307, 1994.

Tabla 3: Ficheros afectados por los cambios descritos en este documento.

Fichero	Módulo	Cambio
<code>heuristic/canoAndMoral/CanoMoralElimination.java</code>	inference	Corrección de dos bugs
<code>heuristic/weightedMinFill/WeightedMinFillInfo.java</code>	inference	Nueva clase
<code>heuristic/lookahead/LookaheadHeuristic.java</code>	inference	Nueva clase abstracta
<code>heuristic/lookahead/LookaheadMinFill.java</code>	inference	Nueva clase
<code>heuristic/lookahead/LookaheadMinCliqueSize.java</code>	inference	Nueva clase
<code>heuristic/lookahead/LookaheadCanoMoral.java</code>	inference	Nueva clase
<code>src/main/java/module-info.java</code>	inference	Exportación de paquetes nuevos
<code>inference/heuristics/HeuristicComparisonTest.java</code>	Testgrajava	Test comparativo con paralelismo
<code>run-heuristic-comparison.sh</code>	(raíz)	Script de ejecución del test